



KARADENİZ TEKNİK ÜNİVERSİTESİ — BİLGİSAYAR BİLİMLERİ

Hafta 1 Quiz

Web'in Temelleri: Ağdan HTTP'ye

Doç. Dr. Tolga Berber · <https://avesis.ktu.edu.tr/tberber>

Öğrenme fazları: 1 • Ön-öğretim 2 • İçerik 3 • Katılım 4 • Değerlendirme 5 • İzleme

Soru 1 (6 puan)

4 • Değerlendirme Hedef 1

TCP/IP katman modelinin basitleştirilmiş dört katmanını (uygulama, taşıma, İnternet, bağlantı) doğru sorumluluklarıyla eşleştiren seçenek hangisidir?

- (a) Uygulama katmanı HTTP/DNS gibi protokolleri taşır; taşıma katmanı TCP/UDP ile port numaralarını yönetir; İnternet katmanı IP adresleme ve yönlendirmeden sorumludur; bağlantı katmanı verinin Ethernet/Wi-Fi gibi fiziksel bir ortamda taşınmasını sağlar.
- (b) Uygulama katmanı IP adreslerini atar; taşıma katmanı HTTP isteklerini işler; İnternet katmanı port numaralarını yönetir; bağlantı katmanı DNS çözümlemesi yapar.
- (c) Bu dört katman, OSI modelinin 7 katmanı ile birebir aynı isimlere ve sorumluluklara sahiptir; TCP/IP modeli yalnızca OSI'nin başka bir adıdır.
- (d) Port numarası İnternet katmanının işidir, IP adresleme ise taşıma katmanının işidir — çünkü ikisi de bir tür "adresleme" kavramıdır.

Soru 2 (6 puan)

4 • Değerlendirme Hedef 1

YG II'de `QTcpSocket::connectToHost(ip, port)` çağırdığınızda IP adresi ve port farklı görevler üstlenir. Aşağıdakilerden hangisi bu görev ayrımını doğru açıklar?

- (a) Port bir programın kalıcı kimliğidir; bir program her zaman aynı portu kullanmak zorundadır ve başka hiçbir program o portu asla kullanamaz.
- (b) IP adresi olmadan yalnızca port numarasıyla da doğru makineye ulaşmak mümkündür; port, IP adresinin yerini tamamen alabilir.
- (c) IP adresi ağdaki doğru makineyi (binayı) bulur; port o makinede çalışan doğru servisi (daireyi/kapıyı) belirler — aynı IP adresinde farklı portlardan birden çok servis aynı anda dinleyebilir.
- (d) Port, yalnızca güvenlik amacıyla kullanılan rastgele bir sayıdır; herhangi bir servisi belirtmez.

Soru 3 (8 puan)

4 • Değerlendirme Hedef 1

Aşağıdaki betik kesiti `https://www.example.com` için çalıştırılıyor ve `curl -v` çıktısından yalnızca "Connected to" ile başlayan satırı süzüyor.

```
1 | CURL_CIKTI=$(curl -sS -v -m 10 "https://${ALAN_ADİ}" -o /dev/null 2>&1)
```

Soru 3 (devamı)

2 | `echo "${CURL_CIKTI}" | grep -i "Connected to"`

Bu betik çalıştığında ekrana basılan satırda göreceğiniz port numarası kaçtır ve bu numara neden betikte hiçbir yerde sabit yazılı olmadığı hâlde hep aynı çıkar?

Soru 4 (10 puan)

4 • Değerlendirme Hedef 1

IP adresi, port ve DNS üçlüsünün, tarayıcının bir web sitesine bağlanması sürecinde birlikte nasıl çalıştığını, YG II’de yazdığınız `connectToHost(ip, port)` çağrısına atıfla kendi cümlelerinizle açıklayınız.

Soru 5 (6 puan)

4 • Değerlendirme Hedef 2

Bir web sunucusunun "statik içerik" ile "dinamik içerik" sunması arasındaki farkı doğru açıklayan seçenek hangisidir?

- (a) Statik içerik her zaman resim dosyalarıdır, dinamik içerik her zaman metin dosyalarıdır; ayırım dosya türüne dayanır.
- (b) Statik içerik, sunucunun diskte hazır duran bir dosyayı olduğu gibi gönderdiği, her istekte aynı kalan cevaptır; dinamik içerik, sunucunun o anki isteğe/duruma göre *anında hesapladığı*, istekten isteğe değişebilen cevaptır.
- (c) Dinamik içerik her zaman rastgele veya gerçek zamanlı üretilmiş anlamına gelir; sunucunun hesapladığı her cevap mutlaka her istekte bir öncekinden farklı çıkar.
- (d) Statik/dinamik ayrımı yalnızca dosyanın boyutuna bağlıdır; büyük dosyalar dinamik, küçük dosyalar statik kabul edilir.

Soru 6 (6 puan)

4 • Değerlendirme Hedef 2

HTTP/HTTPS’in port 80/443 kullanması ile bir ters vekilin (*reverse proxy*) rolü hakkında hangisi doğrudur?

- (a) Tarayıcı bir URL’de port numarasını her zaman açıkça belirtmek zorundadır; varsayılan port diye bir kavram yoktur.
- (b) Port 80/443 kullanmak teknik bir zorunluluktur, HTTP/HTTPS başka hiçbir portta çalışmaz; ters vekil yalnızca bir güvenlik duvarıdır, herhangi bir yönlendirme yapmaz.

Soru 6 (devamı)

- (c) Ters vekil isteği doğrudan veritabanına yönlendirir; uygulama sunucusunu bu süreçte tamamen atlar.
- (d) Port 80/443 teknik bir zorunluluk değil, evrensel bir sözleşmedir (tarayıcı URL'de port belirtmezse bu varsayılanı dener); bir ters vekil, istemciden gelen isteği doğrudan uygulama sunucusuna değil önce kendisine alıp statik dosyalar ile dinamik uygulama arasında yönlendirir.

Soru 7 (8 puan)**4 • Değerlendirme Hedef 2**

Aşağıdaki betik kesiti çalıştırıldığında, STATIK_URL için iki ardışık istek ile DINAMIK_URL için iki ardışık istek karşılaştırılıyor.

```
1 STATIK_URL="https://example.com/"
2 DINAMIK_URL="https://httpbin.org/uuid"
3
4 ISTEK1=$(curl -sS -m 10 "${STATIK_URL}")
5 ISTEK2=$(curl -sS -m 10 "${STATIK_URL}")
6
7 CEVAP1=$(curl -sS -m 10 "${DINAMIK_URL}")
8 CEVAP2=$(curl -sS -m 10 "${DINAMIK_URL}")
```

Beklenen (doğru) davranışta ISTEK1/ISTEK2 ile CEVAP1/CEVAP2 çiftleri arasında ne fark olur ve bu farkın altında yatan neden nedir?

Soru 8 (6 puan)**4 • Değerlendirme Hedef 3**

Modern bir WebApp'in üç katmanlı mimarisinde (sunum, iş, veri) her katmanın sorumluluğunu doğru tanımlayan seçenek hangisidir?

- (a) Üç katman her zaman aynı süreçte (process) çalışmak zorundadır; web'de bunları ayrı makinelere dağıtmak mümkün değildir.
- (b) Sunum katmanı veritabanı sorgularını çalıştırır, iş katmanı yalnızca arayüzü çizer, veri katmanı iş kurallarını uygular.
- (c) Sunum katmanı kullanıcının doğrudan gördüğü/etkileşime girdiği arayüzdür; iş katmanı kuralların uygulandığı mantıktır (ör. "bir koltuk iki kez satılamaz"); veri katmanı bilginin kalıcı tutulduğu yerdir.
- (d) İş katmanı yalnızca veritabanı bağlantısını açıp kapatmakla sorumludur; herhangi bir kural uygulamaz.

Soru 9 (6 puan)**4 • Değerlendirme Hedef 3**

API kavramı ile SPA/SSR ayrımı hakkında hangisi doğrudur?

- (a) API, bir yazılım bileşeninin bir başkasına önceden anlaşılmış bir sözleşmeyle sunduğu arayüzdür (bu ders boyunca API'ler JSON taşır). SPA'da tarayıcı bir kez HTML/JS indirir, sonra yalnızca API'den veri çekip arayüzü JavaScript ile günceller (React); SSR'da her sayfa değişiminde sunucu tam bir HTML sayfası üretip gönderir.
- (b) API yalnızca veritabanı sorgularının başka bir adıdır; SPA her sayfa geçişinde sunucudan

Soru 9 (devamı)

tam bir HTML indirir, SSR ise hiçbir zaman HTML üretmez.

- (c) Bu ders sunucu taraflı şablon (Jinja2 gibi) render kullanacaktır; API'ler doğrudan HTML formatında veri taşıyacaktır.
- (d) SPA ile SSR arasında pratikte fark yoktur; ikisi de her sayfa isteğinde sunucudan tam bir HTML sayfası indirir.

Soru 10 (6 puan)

4 • Değerlendirme Hedef 3

Etkinlik/Bilet Sisteminde "seçilen koltuğun hâlâ boş olup olmadığının kontrol edilmesi" işlemi üç katmandan (sunum/iş/veri) hangisinde yer almalıdır?

- (a) Sunum katmanında — React bileşeni bu kontrolü doğrudan kendisi yapmalı, sunucuya hiç sormamalıdır.
- (b) İş katmanında — bu, koltuğun durumuna göre bir karar/kural uygulanmasıdır; sunum katmanı yalnızca göstergeyi çizer, veri katmanı yalnızca kaydı saklar.
- (c) Veri katmanında — SQLite veritabanı bu kontrolü kendiliğinden yapar, ayrı bir mantık gerekmez.
- (d) Üç katmanın hiçbirinde — bu kontrol tarayıcının kendi önbelleğinde otomatik olarak yapılır.

Soru 11 (10 puan)

4 • Değerlendirme Hedef 3

Bu dersin teknoloji yığını (React, FastAPI, SQLite) üç katmanlı mimariye yerleştiriniz; ayrıca "aynı koltuğun iki kez satılmaması" gibi bir iş kuralının neden sunum katmanında (React tarafında) değil, iş katmanında (FastAPI tarafında) uygulanması gerektiğini açıklayınız.

Soru 12 (6 puan)

4 • Değerlendirme Hedef 4

HTTP metotlarının (GET, POST, PUT/PATCH, DELETE) niyetini doğru eşleştiren seçenek hangisidir?

- (a) GET isteği her zaman bir gövde (body) taşır; POST isteği hiçbir zaman gövde taşımaz.

Soru 12 (devamı)

- (b) GET veri siler, POST veri okur, PUT yeni kaynak oluşturur, DELETE günceller.
- (c) Dört metodun da işlevi aynıdır; hangisinin kullanıldığı sunucu için hiçbir fark oluşturmaz.
- (d) GET var olan bir veriyi okur (değiştirmez); POST yeni bir veri/kaynak oluşturur; PUT/PATCH var olan bir kaynağı günceller; DELETE bir kaynağı siler.

Soru 13 (8 puan)**4 • Değerlendirme** Hedef 4

Aşağıdaki ham HTTP isteğini inceleyiniz.

```
>_ Terminal
GET /etkinlikler HTTP/1.1
Host: biletsistemi.example.com
User-Agent: curl/8.4.0
Accept: application/json
```

- (a) İstek satırı hangisidir; metod ve yol nedir? (b) Kaç başlık satırı vardır ve her biri ne bildirir? (c) Bu istekte bir gövde (body) var mıdır, neden?

Soru 14 (8 puan)**4 • Değerlendirme** Hedef 4

Aşağıdaki Python kodu bir HTTP GET isteğini elle bir dizge olarak kurar.

```
1 HOST = "example.com"
2 istek = (
3     f"GET / HTTP/1.1\r\n"
4     f"Host: {HOST}\r\n"
5     f"User-Agent: hafta01-soket-istemci\r\n"
6     f"Accept: text/html\r\n"
7     f"Connection: close\r\n"
8     f"\r\n"
9 )
10 soket.sendall(istek.encode("ascii"))
```

soket.sendall(...) satırı sunucuya tam olarak hangi ham metni (satır satır) gönderir? Bu istekte Connection: close başlığı unutulsaydı ne olurdu?

Soru 15 (10 puan)**4 • Değerlendirme** Hedef 4

Bir HTTP GET isteğinde gövdenin (body) genelde neden boş olduğunu, buna karşılık bir POST isteğinde neden dolu olabileceğini açıklayınız; ayrıca 4xx ve 5xx durum kodu ailelerinin bir hatanın kaynağını (istemci mi, sunucu mu) nasıl ayırt etmenize yardımcı olduğunu belirtiniz.

Soru 15 (devamı)
